

Chapter 5

Implementation

5.1 Introduction

This chapter explains how **VisionSafe360** was implemented as a complete AI-based workplace safety monitoring system. The goal of the implementation was to build a practical solution capable of detecting safety hazards in real time using computer vision and machine learning.

VisionSafe360 was designed to monitor industrial environments through live video streams and automatically detect dangerous situations such as missing personal protective equipment (PPE), worker falls, unsafe proximity between workers and vehicles, and harmful ergonomic posture. The system combines AI models, edge processing, backend services, and a web dashboard to provide a complete safety monitoring platform.

This chapter covers the implementation phases, technologies used, system architecture, module development, and challenges faced during development.

5.2 Implementation Overview

The implementation of VisionSafe360 was completed in several phases. The first phase focused on identifying the main workplace hazards that the system should detect. After studying industrial safety requirements, four major hazards were selected: PPE violations, worker falls, unsafe forklift proximity, and poor ergonomic posture.

The second phase involved collecting and preparing datasets for model training. Different datasets were used for PPE detection, forklift detection, and human posture analysis. Data preprocessing and annotation were performed to improve model quality and detection accuracy.

The third phase focused on AI model development. Multiple computer vision models were trained and tested for object detection, tracking, and pose estimation. These models formed the core of the hazard detection engine.

In the fourth phase, backend services were developed to manage alerts, incidents, analytics, and communication between system components. A dashboard was then built to visualize live monitoring and safety reports.

Finally, all modules were integrated to produce the final prototype of VisionSafe360. The final deliverable is a complete AI-powered industrial safety system capable of monitoring live video streams and generating real-time alerts.

5.3 Development Milestones

The development process followed a structured sequence of milestones to ensure smooth progress.

The first milestone involved defining project requirements and system architecture. During this stage, the overall pipeline was designed, including AI processing, backend communication, and dashboard visualization.

The second milestone focused on dataset preparation and model selection. Different AI models were evaluated based on speed and accuracy to ensure suitability for real-time deployment.

The third milestone involved implementing hazard detection modules. Each safety feature was developed as a separate module to maintain modularity and scalability.

The fourth milestone focused on backend and frontend development. APIs were created to manage system communication, while the dashboard provided an interface for monitoring and analytics. The final milestone involved system integration and prototype completion.

5.4 System Architecture Implementation

VisionSafe360 was implemented using a layered architecture to ensure scalability and maintainability. The system consists of four main layers: video input, edge AI processing, backend services, and dashboard visualization.

The **video input layer** receives data from CCTV cameras, webcams, IP cameras, or uploaded video files. These video streams act as the primary input source for hazard detection.

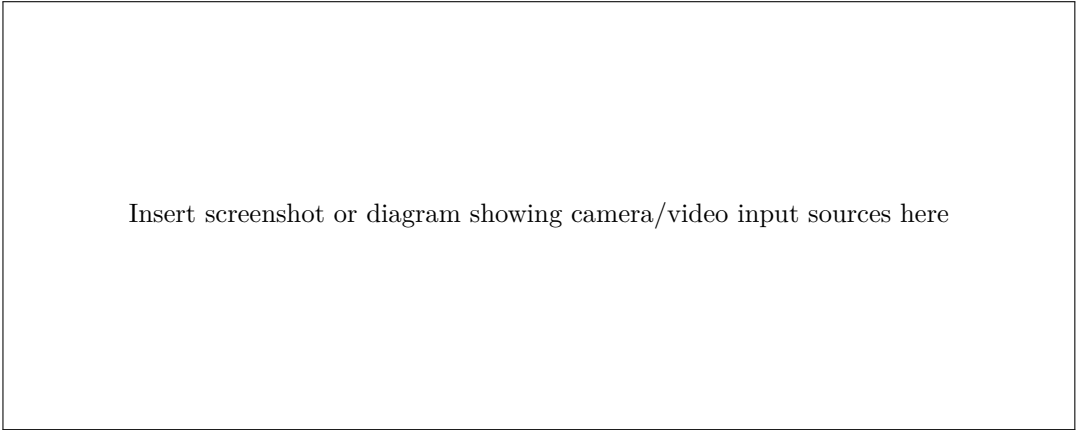


Figure 5.1: Video input pipeline

The **edge AI processing layer** performs real-time inference on incoming frames. This layer runs AI models for object detection, pose estimation, object tracking, and hazard analysis. Running inference on the edge reduces latency and allows faster response.

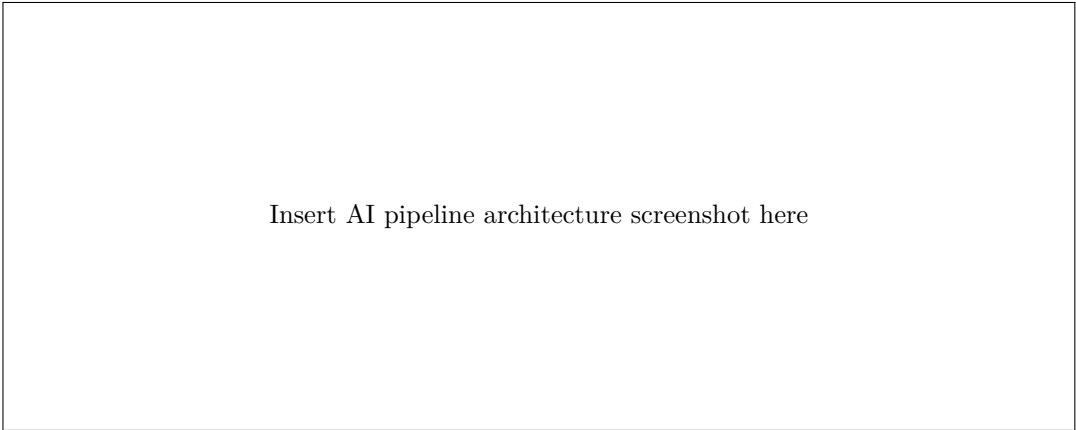


Figure 5.2: Edge AI processing pipeline

The **backend layer** handles event logging, alert management, API communication, and analytics processing. It acts as the bridge between AI inference and dashboard visualization.

The **dashboard layer** provides the user interface for live monitoring, incident tracking, safety analytics, and system configuration.

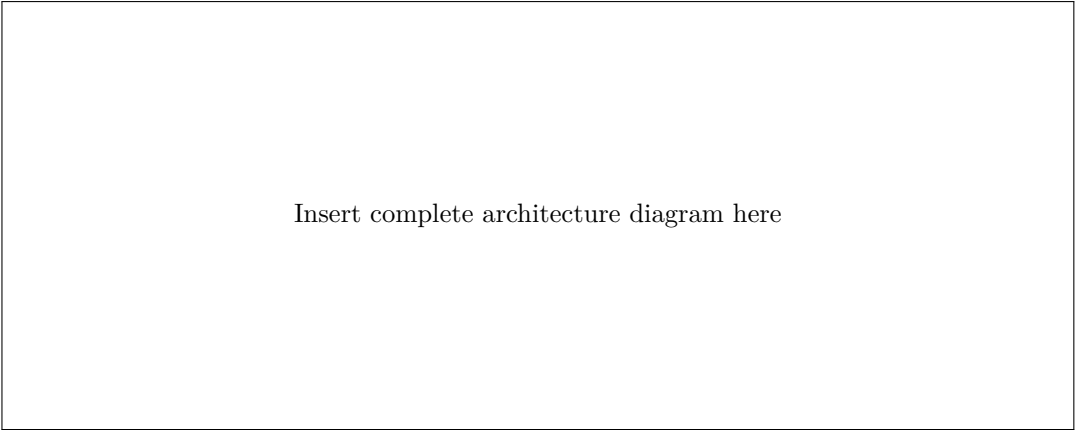


Figure 5.3: Complete VisionSafe360 architecture

5.5 Technologies and Tools Used

VisionSafe360 was built using modern AI and web development technologies to ensure performance, scalability, and ease of development. Table 5.1 summarizes the main technologies and tools used.

Table 5.1: Technologies and tools used in VisionSafe360

Category	Technologies
Programming Languages	Python, TypeScript
AI Framework	PyTorch
Detection Models	Ultralytics YOLO
Computer Vision	OpenCV
Backend	FastAPI
Frontend	React
UI Styling	Tailwind CSS
Database Layer	SQLAlchemy
Caching	Redis
Deployment	Docker

These tools were selected because they provide strong support for real-time AI applications and modern web systems.

5.6 Hardware Environment

The project was developed and tested using a laptop with GPU acceleration for AI inference and model execution. Table 5.2 lists the hardware environment.

Table 5.2: Hardware environment used for development and testing

Component	Specification
CPU	Intel Core i5-13420H
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
RAM	16 GB

This hardware was sufficient to achieve near real-time inference while running multiple AI modules simultaneously.

5.7 PPE Detection Implementation

The PPE detection module was developed to monitor worker compliance with required safety equipment. The system checks whether workers are wearing essential protective equipment such as helmets and safety vests.

A YOLO-based detection model was used because of its balance between speed and accuracy for real-time detection. The model was trained using the SH17 PPE dataset. Table 5.3 presents the PPE dataset and model details.

Table 5.3: PPE dataset and model details

Item	Value
Dataset	SH17 PPE Dataset
Images	8,099
Classes	17
Instances	75,994
Resolution	640×640
Model	YOLOv9-e

Each video frame is processed to detect PPE items. If required equipment is missing for several consecutive frames, the system generates a violation alert to reduce false positives caused by temporary occlusion.

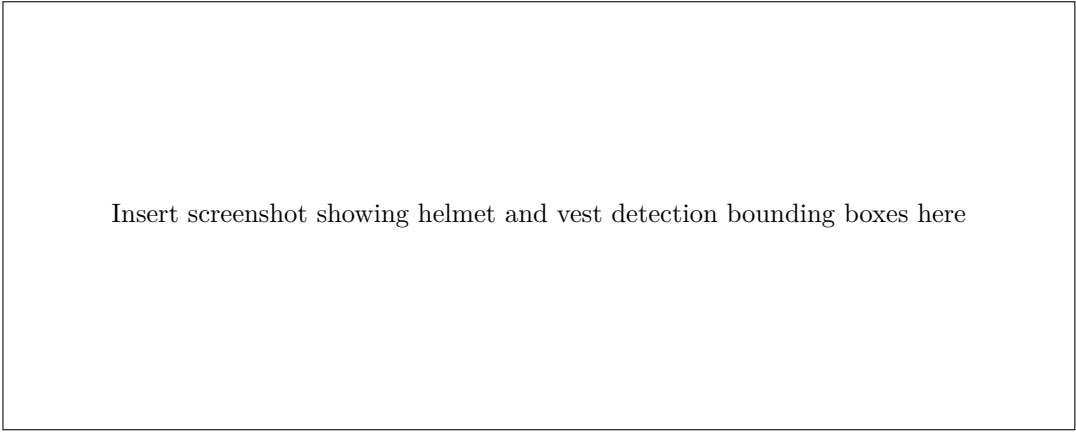


Figure 5.4: PPE detection output

5.8 Forklift Proximity Detection Implementation

This module detects unsafe distances between workers and moving forklifts. Its purpose is to reduce collision risks inside industrial environments.

The system first detects workers and forklifts using object detection. After detection, object tracking maintains identity across frames. Distance estimation is then used to determine whether a worker enters a dangerous zone around a forklift.

The module uses a YOLOv8m model trained on a forklift dataset. Table 5.4 summarizes the forklift proximity model configuration.

Table 5.4: **Forklift proximity model configuration**

Item	Value
Model	YOLOv8m
Training Images	2,468
Validation Images	500
Epochs	200
Batch Size	16
Resolution	640×640

If unsafe proximity persists for more than one second, the system generates an alert.

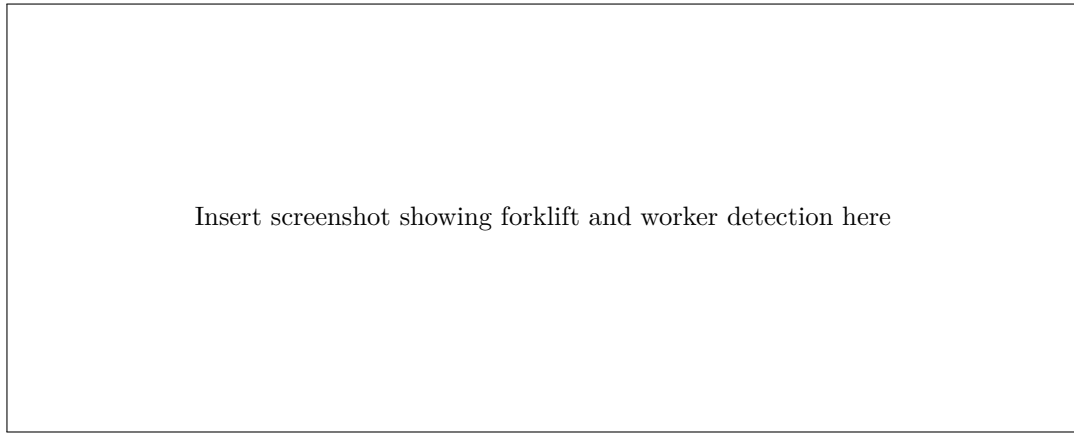


Figure 5.5: Forklift detection output

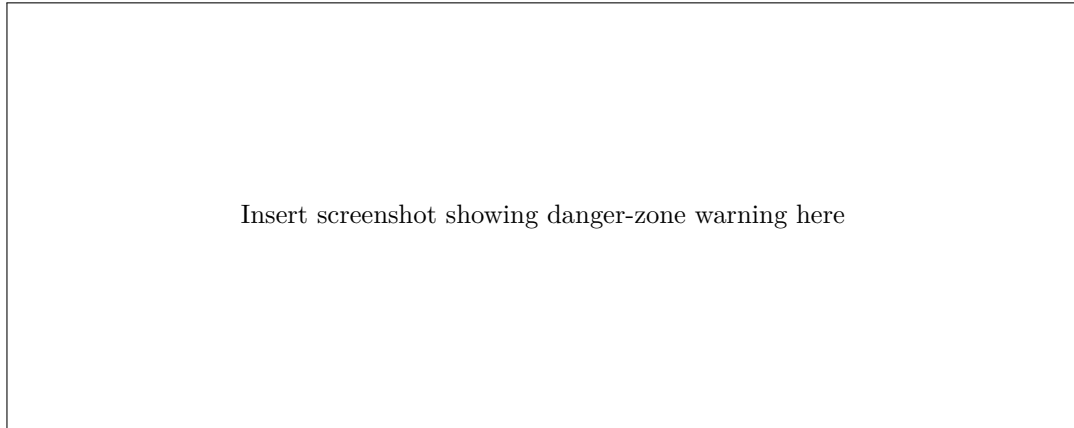


Figure 5.6: Unsafe proximity alert

5.9 Fall Detection Implementation

The fall detection module identifies workplace fall incidents in real time. Falls are considered critical events because delayed response can significantly increase injury severity.

Instead of using sequence-based models such as LSTM, VisionSafe360 uses a pose-aware rule-based approach. Human pose estimation is used to track body movement and posture changes across frames. The fall detection engine uses four states:

1. Normal

2. Fall Candidate
3. Fall Confirmed
4. Fall Recovered

The system analyzes body geometry, motion direction, and posture transitions to determine whether a fall has occurred. A fall is only confirmed if abnormal posture persists for approximately 2.5 seconds. This reduces false alarms caused by temporary bending or crouching.

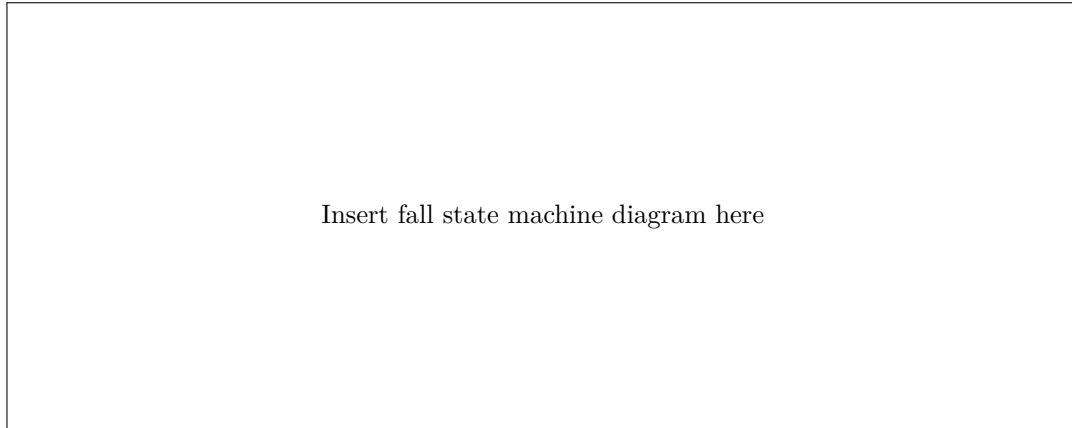


Figure 5.7: Fall detection state machine

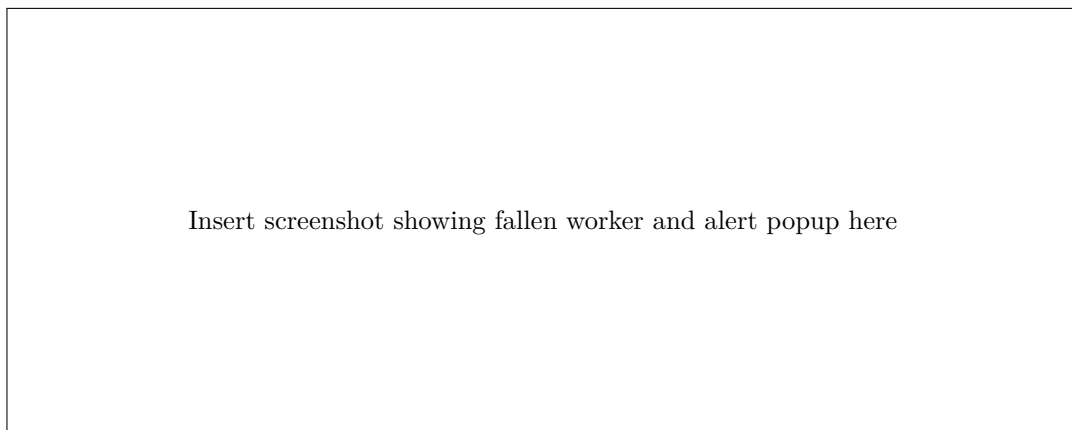


Figure 5.8: Fall detection alert

This design provides reliable fall detection while maintaining low computational cost.

5.10 Ergonomic Posture Analysis Implementation

The ergonomic posture analysis module helps detect unsafe body posture that may cause long-term musculoskeletal injuries.

The system uses body keypoints extracted from pose estimation to calculate joint angles. These angles are used to compute ergonomic risk scores based on industry-standard assessment methods.

Two scoring systems were implemented:

- RULA (Rapid Upper Limb Assessment)
- REBA (Rapid Entire Body Assessment)

Alert thresholds were defined as shown in Table 5.5.

Table 5.5: **Ergonomic posture alert thresholds**

Metric	Threshold
RULA Alert	≥ 5
REBA Alert	≥ 7
Critical Risk	RULA = 7 or REBA ≥ 11

To reduce unstable scoring caused by pose jitter, smoothing techniques were applied.

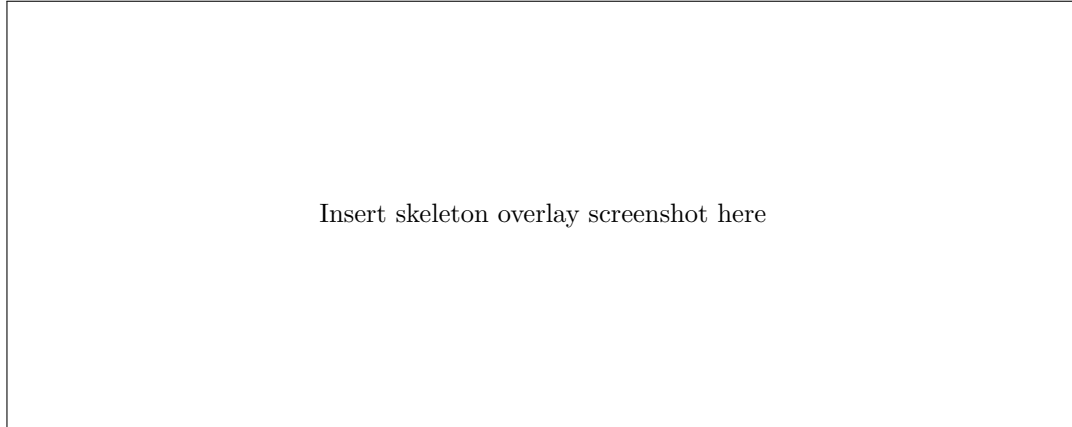


Figure 5.9: Posture skeleton detection

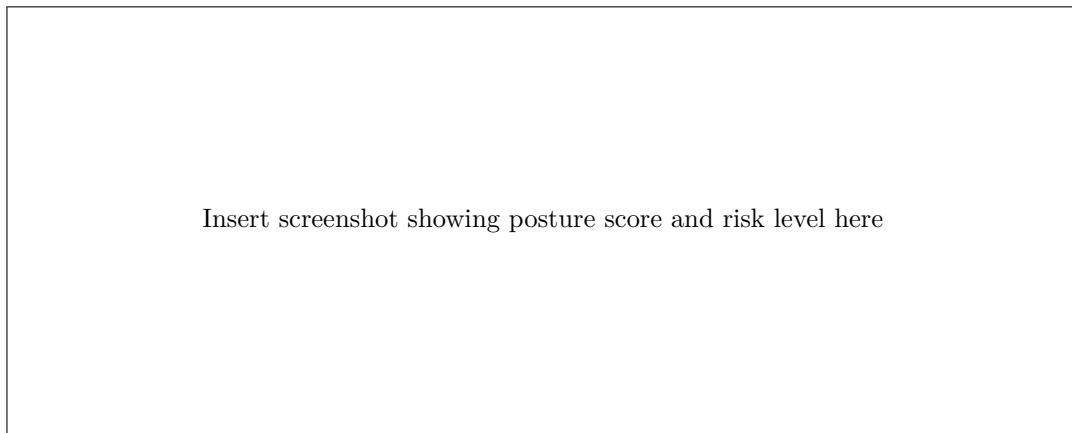


Figure 5.10: Ergonomic risk alert

This module helps identify repeated harmful movements before they lead to injuries.

5.11 Tracking and Event Persistence

Object tracking plays an important role in VisionSafe360. The system uses tracking to maintain object identity across consecutive frames.

Tracking improves system reliability by reducing duplicate detections and enabling temporal analysis. It also helps maintain stable worker and vehicle IDs during movement.

This improves fall detection consistency, ergonomic scoring stability, and alert persistence logic. Without tracking, temporary detection loss could generate unnecessary alerts.

5.12 Backend Implementation

The backend was developed using FastAPI. It serves as the central communication layer between AI modules and the dashboard.

The backend handles incident logging, alert management, analytics processing, API communication, and WebSocket-based real-time updates.

Whenever a hazard is detected, the AI engine sends event data to the backend, where incidents are stored and processed before being displayed to users.

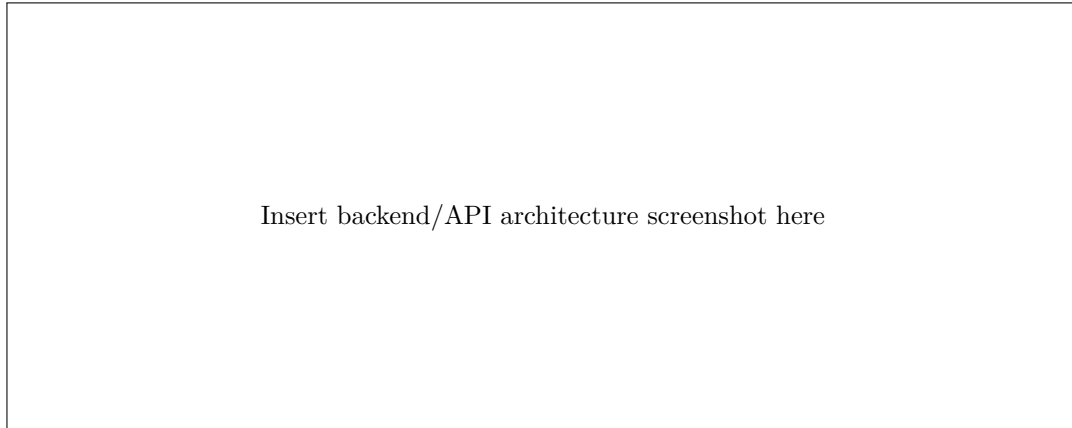


Figure 5.11: Backend structure

The backend ensures smooth communication across all system components.

5.13 Dashboard Implementation

The dashboard was implemented using React and TypeScript.

It provides a complete interface for safety monitoring and analytics. Supervisors can view live streams, inspect incidents, monitor trends, and review ergonomic reports.

Main dashboard sections include:

- Dashboard Home
- Live Monitoring
- Alerts
- Incident Management
- Reports
- Ergonomics Analytics
- Camera Management
- System Configuration

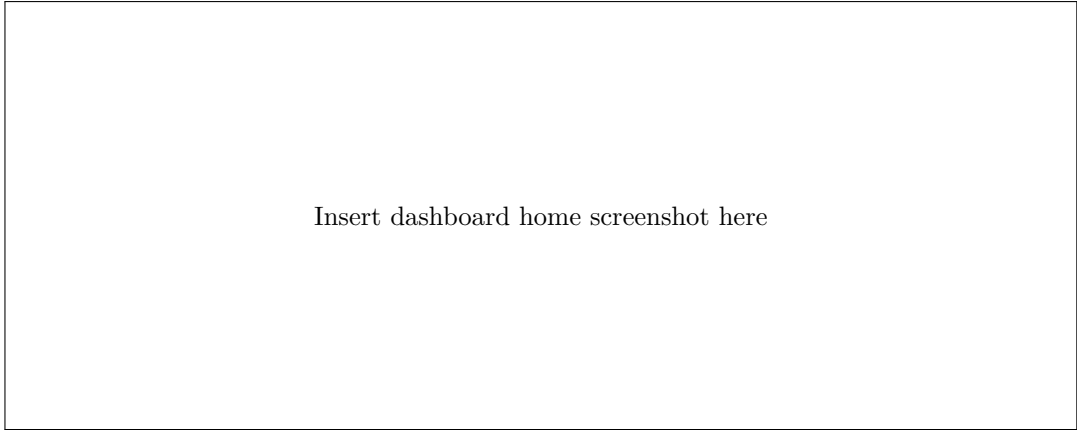


Figure 5.12: Main dashboard

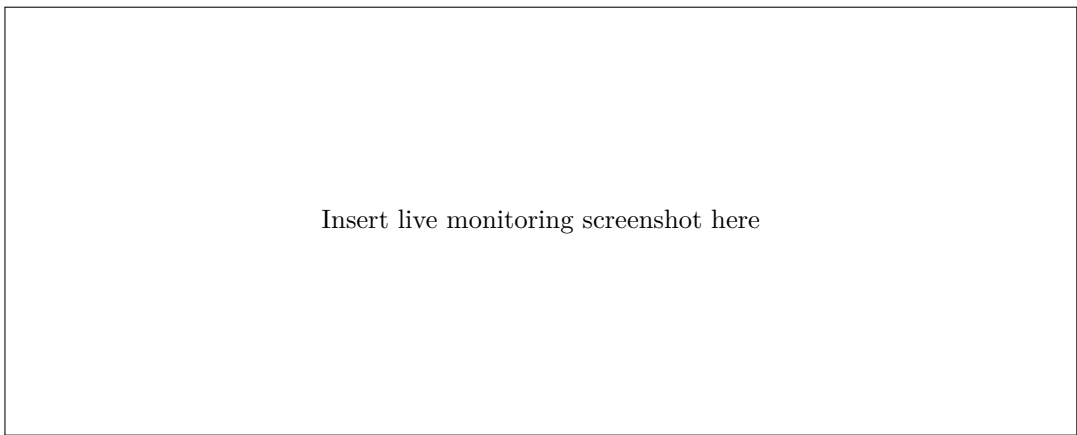


Figure 5.13: Live monitoring page

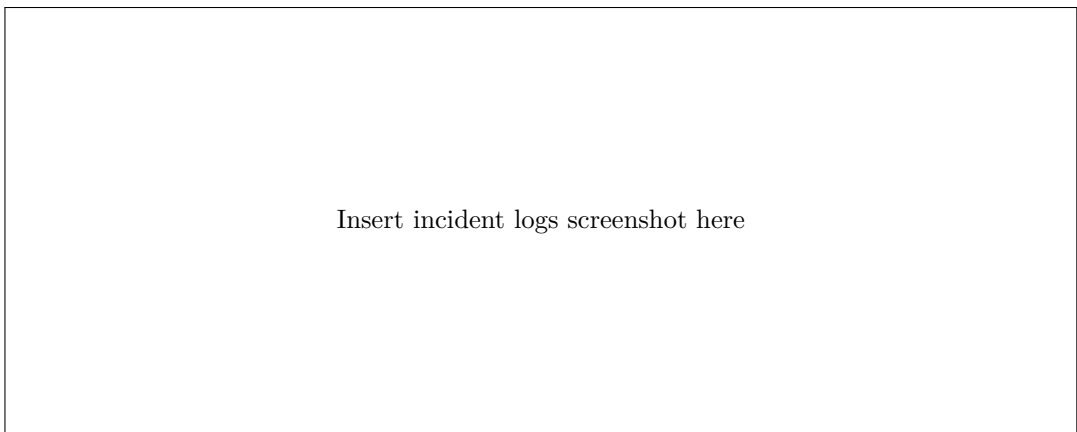


Figure 5.14: Incident management page

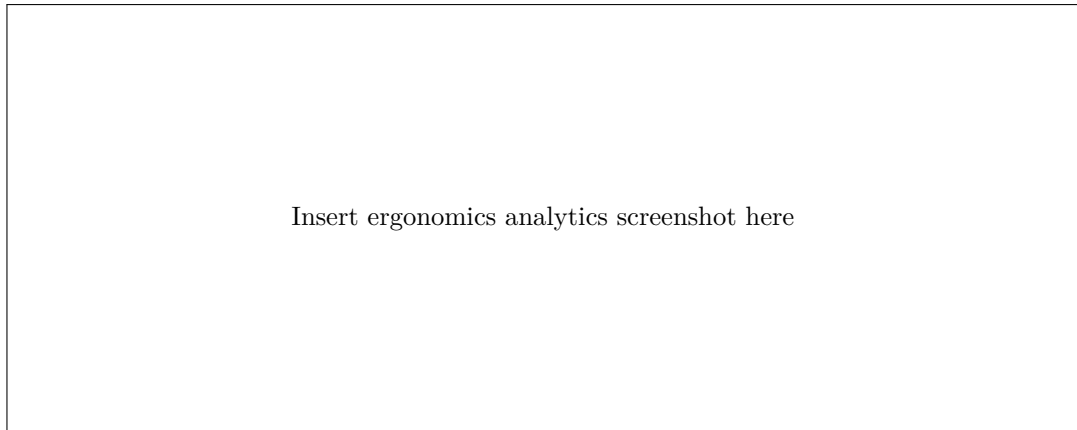


Figure 5.15: Ergonomics dashboard

The dashboard makes safety information easier to understand and act on.

5.14 Challenges Encountered

Several technical challenges were faced during implementation.

One challenge was false-positive alerts caused by temporary occlusion. For example, a worker's helmet might become temporarily hidden due to camera angle. This problem was reduced by using alert persistence logic instead of triggering alerts immediately.

Another challenge involved lighting conditions. Poor illumination reduced detection confidence in some scenarios. Confidence thresholds and preprocessing helped improve robustness.

Pose estimation also introduced occasional skeleton jitter, which affected posture scoring. This issue was reduced using smoothing techniques.

Camera perspective was another challenge for proximity estimation because depth information is limited in 2D video. Calibration and threshold tuning improved reliability.

5.15 Summary

In this chapter, the implementation of VisionSafe360 was presented in detail. The project successfully integrated AI models, real-time hazard analysis, backend services, and dashboard visualization into a unified workplace safety monitoring system.

The final system provides real-time detection of PPE violations, falls, unsafe proximity, and ergonomic risks while maintaining practical performance for deployment. The implementation demonstrates how AI can significantly improve workplace safety through continuous monitoring and fast hazard response.